Chapter 4
Learning and Evaluating Cross-Sectional Return Predictors
Asset Pricing & Machine Learning in Finance

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 2026

Contents

Model Architecture, Training & Evaluation

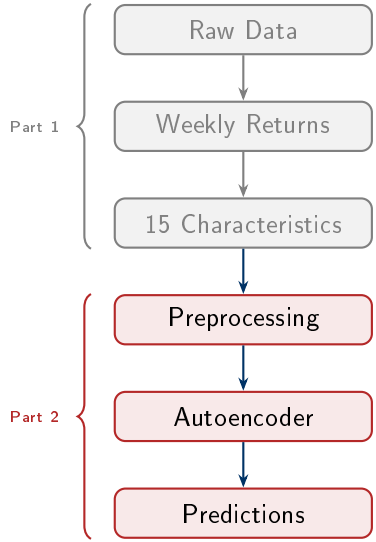
Where We Are in the Project

Part 1 (Previous Chapter)

- Downloaded prices & metadata
- Constructed weekly returns
- Engineered 15 firm-level characteristics
- Stored everything in HDF5

Part 2 (This Chapter)

- Load & preprocess the dataset
- Build the conditional autoencoder in Keras
- Walk-forward cross-validation
- Hyperparameter search & evaluation
- Generate out-of-sample predictions



1. Setup and Imports (Cells 0–8)

The notebook begins by importing all the necessary libraries. The main ones are:

- **TensorFlow/Keras**: for building and training the neural network
- **Pandas and NumPy**: for data manipulation
- **Scikit-learn** (`quantile_transform`): for rank normalization
- **SciPy** (`spearmanr`): for computing the Information Coefficient
- **Matplotlib and Seaborn**: for plotting

Feature Selection and Setup

Then, the list of 15 features to be used as input to the beta network is defined:

Python

```
 characteristics = ['beta', 'betasq', 'chmom', 'dolvol', 'idiovol', 'ill',
'indmom',
        'maxret', 'mom12m', 'mom1m', 'mom36m', 'mvel', 'retvol', 'turn']
```

These are the same variables constructed in the previous notebook (Part 1, data preparation): 6 price trend factors, 5 liquidity factors, and 4 risk factors — all computed using rolling windows on historical data.

Finally, the path where all results will be saved is set, and the random seed is fixed at 42 for reproducibility.

2. Data Loading (Cells 9–14)

The notebook loads weekly returns from the HDF5 file produced during the data preparation phase:

Python

```
data = (pd.read_hdf(results_path / 'autoencoder_reloaded.h5', 'returns')
    .stack(dropna=False)
    .to_frame('returns')
    .loc[idx['1993':, :], :])
```

The result is a DataFrame with a MultiIndex (`date, ticker`) and one column `returns`. The structure is in "long" format: each row corresponds to one asset in a specific week.

The choice of starting from 1993 is due to the fact that many features require 3-year rolling windows, so data before 1993 do not have sufficient history for computation.

3. Exploratory Analysis of a Single Asset (Cells 15–18)

The notebook isolates Apple (AAPL) returns and visualizes them in two ways: a time series plot of weekly returns and a histogram of their distribution.

This is a simple sanity check — it allows us to visually verify that the data are reasonable, without anomalous spikes or obvious gaps.

4. Feature Merging (Cells 19–25)

The 15 features are loaded from the HDF5 file and added to the main DataFrame. In the file, each feature is stored with a key starting with `factor/` (e.g., `factor/beta`, `factor/mom12m`):

Python

```
with pd.HDFStore(results_path / 'autoencoder_reloaded.h5') as store:
    keys = [k[1:]  for k in store.keys() if k[1:].startswith('factor/')]
    for key in keys:
        data[key.split('/')[-1]] = store[key].squeeze()
```

After this step, the DataFrame `data` has one row per (week, asset) pair and 16 columns: `returns` plus the 15 features.

The notebook then updates the `characteristics` list by directly reading the DataFrame columns (excluding `returns`).

5. Target Creation: Forward Returns (Cell 26)

This is a fundamental line:

Python

```
data['returns_fwd'] = data.returns.unstack('ticker').shift(-1).stack()
```

What it does in detail: it takes returns, pivots them into wide format (date $\times$ ticker), shifts everything up by one row (so that next week's return is aligned with the current week), and then converts back to long format.

The result is a new column `returns_fwd`, where for each row corresponding to week t and asset i, we find the return at time $t + 1$.

This is the model's target: the network learns to use today's features to predict tomorrow's return. Without this shift, the model would attempt to "predict the present," which has no practical value.

6. Data Coverage Analysis (Cells 29–32)

The notebook computes and visualizes:

- How many assets are available in each week (distribution of the number of observations per date)
- How many observations each feature has over time (to verify that there are no features with too many missing values)

This serves as a quality check: if a feature had very few observations in certain weeks, it could be unreliable and should be treated differently.

7. Rank Normalization (Cell 34)

This is one of the most important steps:

Python

```
data.loc[:, characteristics] = (
    data.loc[:, characteristics]
        .groupby(level='date')
        .apply(lambda x:  pd.DataFrame(
            quantile_transform(x, copy=True, n_quantiles=x.shape[0]),
            columns=characteristics,
            index=x.index.get_level_values('ticker')
        ))
        .mul(2).sub(1)
)
```

For each week separately (`groupby(level='date')`), each feature is transformed into its empirical quantiles via `quantile_transform`. This produces values in $[0, 1]$, where 0 corresponds to the lowest value and 1 to the highest. Then, `.mul(2).sub(1)` rescales the interval from $[0, 1]$ to $(-1, 1)$.
The result is that all 15 features become comparable, bounded in the same range, free of outliers, and with a uniform cross-sectional distribution each week. This is essential for stable neural network training.

9. Model Architecture (Cells 42–61)

The notebook builds a conditional autoencoder in Keras. Let us examine each component.

Inputs (Cell 49):

Python

```
input_beta = Input((n_tickers, n_characteristics), name='input_beta')
input_factor = Input((n_tickers,), name='input_factor')
```

Two separate input layers: `input_beta` receives a matrix (all assets $\times$ 15 features) for each week. `input_factor` receives a vector of returns (one value per asset) for each week.

Beta Network (Cell 51)

Python

```
hidden_layer = Dense(units=8, activation='relu', name='hidden_layer')(input_beta)
batch_norm = BatchNormalization(name='batch_norm')(hidden_layer)
output_beta = Dense(units=n_factors, name='output_beta')(batch_norm)
```

Three steps in sequence. The Dense layer with 8 neurons and ReLU transforms the 15 features into 8 non-linear features, applying the same weights to each asset (weight sharing).

Batch Normalization standardizes these activations to stabilize training.

The final Dense layer (linear, no activation) produces K factor loadings for each asset.

Output shape: (batch, n_tickers, K).

Factor Network (Cell 53)

Python

```
output_factor = Dense(units=n_factors, name='output_factor')(input_factor)
```

A single linear layer that takes returns of all assets and combines them into K latent factors.

Output shape: (batch, K).

Dot Product (Cell 55)

Python

```
output = Dot(axes=(2,1), name='output_layer')([output_beta, output_factor])
```

For each asset, this computes the dot product between its beta vector (K values) and the factor vector (K values), producing the predicted return.

The argument `axes=(2,1)` instructs Keras to contract dimension 2 of `output_beta` (the K factors) with dimension 1 of `output_factor` (also K).

Output shape: (batch, `n_tickers`) — one predicted return for each asset in each week.

Model Compilation (Cell 57)

Python

```
 model = Model(inputs=[input_beta, input_factor], outputs=output)
model.compile(loss='mse', optimizer='adam')
```

The loss function is MSE (Mean Squared Error):

$$\frac{1}{N}\sum_{i}(r_{i,t} - \hat{r}_{i,t})^2$$

The optimizer is Adam, a variant of SGD with an adaptive learning rate.

Helper Function (Cell 59): `make_model(hidden_units, n_factors)` encapsulates the entire architecture into a parametric function, allowing easy creation of models with different hyperparameter combinations.

10. Cross-Validation Setup (Cells 62–68)

Time Series CV (Cell 67):

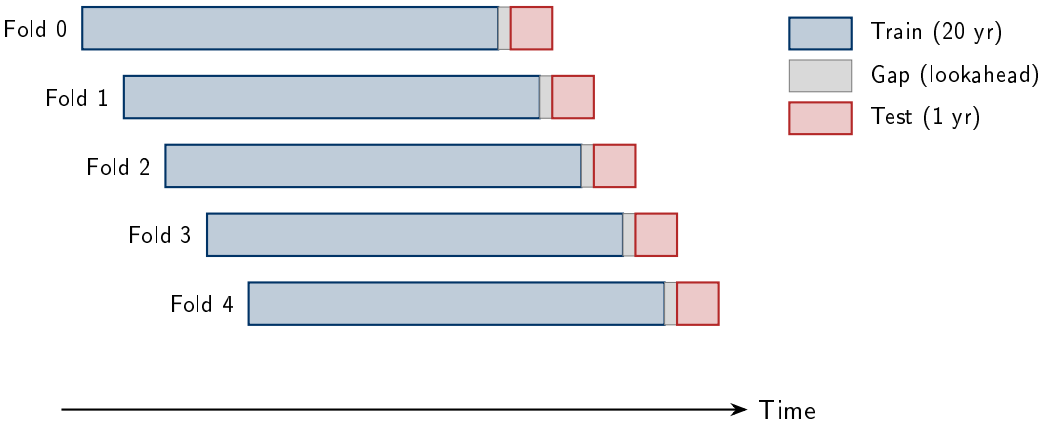
Python

```
cv = MultipleTimeSeriesCV(n_splits=5,
     train_period_length=20*YEAR,
     test_period_length=1*YEAR,
     lookahead=2)
```

Creates a 5-fold walk-forward setup: 20 years of training, a 2-week gap, and 1 year of testing. The windows move forward in time at each fold.

Data Splitting Function (Cell 68): `get_train_valid_data` takes training and validation indices and returns six objects: for each of the two sets (train and validation), it produces features as 3D tensors (weeks $\times$ assets $\times$ 15), returns as 2D matrices (weeks $\times$ assets), and forward returns as targets.

Time-Series Cross-Validation Design



Time-Series Cross-Validation Design

MultipleTimeSeriesCV Parameters

Parameter	Value	Meaning
n_splits	5	Number of walk-forward folds
train_period_length	$20 \times 52 = 1040$ weeks	Training window
test_period_length	$1 \times 52 = 52$ weeks	Validation window
lookahead	2	Gap to prevent information leakage

11. Grid Search and Training (Cells 69–80)

Hyperparameter Grid (Cell 70):

Python

```
 factor_opts = [2, 3, 4, 5, 6]
unit_opts = [8, 16, 32]
```

In total, $5 \times 3 = 15$ combinations are tested.

Main Training Loop (Cell 79)

For each combination (`units`, `n_factors`):

1. Create a new model
2. For each CV fold:
   - Split the data into training and validation sets
   - Train the model incrementally for 250 epochs
   - After each epoch, generate predictions on the validation set
   - Compute two metrics:
     - **Pooled IC** (`r0`): Spearman correlation between all predictions and all realized returns, pooled together
     - **IC by date** (`r1_by_date`): Spearman correlation computed separately for each week, then summarized using mean, median, and standard deviation
   - Save the results

Incremental Training in Cross-Validation (Cell 79)

The relevant code is:

Python

```
model = make_model(hidden_units=units, n_factors=n_factors)
for fold, (train_idx, val_idx) in enumerate(cv.split(data)):
    X1_train, X2_train, y_train, X1_val, X2_val, y_val = get_train_valid_data(...)
    for epoch in range(250):
        model.fit([X1_train, X2_train], y_train,
                  batch_size=batch_size,
                  epochs=epoch + 1,
                  initial_epoch=epoch,
                  verbose=0)
        # compute IC on validation set
        y_pred = model.predict([X1_val, X2_val])
        # ... save metrics for this epoch
```

The key lies in the pair of parameters `epochs=epoch+1` and `initial_epoch=epoch`. Let us examine what happens during the first iterations of the loop.

How Incremental Training Works

When **epoch = 0**: Keras trains from **initial_epoch=0** to **epochs=1**, that is, it performs a single pass over the entire training set. The weights start from their random initialization and are updated once. Then the code exits `model.fit`, computes the IC on the validation set, and saves it.

When **epoch = 1**: Keras trains from **initial_epoch=1** to **epochs=2**. The weights are **not reset** — they start exactly from where they stopped at the end of the previous iteration. Another pass is performed on the training data, and the weights are updated further. Then the IC is computed again.

When **epoch = 2**: training resumes from the current state, performs one more pass, and continues in the same way up to epoch 249.

Why Not Simply Use `model.fit(epochs=250)`?

Because you would lose crucial information. Imagine that the IC on the validation set increases up to epoch 70 and then starts to decline. If you only evaluate at epoch 250, you would observe a mediocre IC and miss the fact that the model performed much better 180 epochs earlier. Incremental training provides a full picture of the model's evolution over time.

To better understand, think of training like baking a cake in the oven. Incremental training is like opening the oven every minute, checking the cake, and noting its condition. In the end, you can say: "the cake was perfect at minute 35". If you only checked at the end (minute 120), you might find a burnt cake without knowing when it was at its best.

Incremental Training and Information Coefficient

Training is incremental: `model.fit(epochs=epoch+1, initial_epoch=epoch)` advances training one epoch at a time without resetting the weights, allowing metrics to be tracked at each step.

The Information Coefficient (IC) measures the model's ability to correctly rank assets: if the model assigns higher expected returns to assets that actually perform better, the IC is positive.

In quantitative practice, an average daily IC of 0.03–0.05 is already considered good.

12. Results Evaluation (Cells 81–96)

The notebook loads all scores saved during cross-validation and analyzes them.

Aggregation Across Folds (Cell 87):

Python

```
avg = (scores.groupby(['n_factors', 'units', 'epoch'])[columns_to_average]
    .mean().reset_index())
```

For each combination (`n_factors`, `units`, `epoch`), it computes the average IC across the 5 folds. This reduces fold-to-fold noise.

Top-5 Epochs per Configuration (Cell 91): For each hyperparameter pair, it selects the 5 epochs with the highest median daily IC.

Visualization and Key Findings

Visualization (Cell 95): A 5×2 plot shows the learning curves (IC vs. epoch) for all combinations. The top row displays pooled IC, while the bottom row shows median daily IC (more robust). Colored lines distinguish 8, 16, and 32 hidden units.

Key Findings (Cell 96):

- 32 hidden units tend to produce unstable or worse results — excessive capacity leads to overfitting
- IC typically peaks between epochs 40 and 100, then declines
- The configuration with **4 factors and 16 hidden units** is the most consistently positive in terms of median daily IC
- 5 factors with 8 units can yield higher pooled IC, but require more aggressive early stopping

13. Final Predictions Generation (Cells 97–103)

With the selected hyperparameters (4 factors, 16 units), the notebook generates the final predictions:

Python

```
for epoch in range(50, 100):
    for fold, (train_idx, val_idx) in enumerate(cv.split(data)):
        model = make_model(...)
        model.fit(..., epochs=epoch)
        predictions = model.predict(...)
```

For each epoch from 50 to 99, and for each fold, a new model is trained from scratch for exactly that number of epochs and then used to predict on the validation set. In total, $50 \times 5 = 250$ prediction sets are generated, concatenated, and averaged.

This ensemble averaging strategy leverages diversity across models (different initializations and training lengths) to produce more stable and robust predictions.

Difference with Cross-Validation (Cell 100)

In the final stage, the strategy is completely different:

Python

```
for epoch in range(50, 100):
    for fold, (train_idx, val_idx) in enumerate(cv.split(data)):
        model = make_model(...)   # NEW model at each iteration
        model.fit(..., epochs=epoch)  # train from scratch for 'epoch' epochs
        predictions = model.predict(...)
```

Here, `make_model(...)` is inside both loops. Each time, a model with new random weights is created, trained from scratch for a fixed number of epochs, and then used for prediction.

There is nothing incremental — this results in $50 \times 5 = 250$ completely independent training runs.

What Happens in Practice

The loop creates **50 different models** (one for each epoch from 50 to 99), and for each of them repeats the procedure across all 5 folds. In detail:

When `epoch = 50`: a new model is created with randomly initialized weights, trained from scratch for exactly 50 epochs, and then used to generate predictions on the validation set.

When `epoch = 51`: another new model is created (with different random weights), trained from scratch for 51 epochs, and predictions are generated.

When `epoch = 52`: same procedure, trained for 52 epochs. This continues up to `epoch = 99`.

At the end (Cell 101), all predictions are concatenated and averaged:

Python

```
predictions_combined = pd.concat(predictions, axis=1).sort_index()
```

Why Do This? The Ensemble Idea

From the cross-validation phase, it emerged that the model with 4 factors and 16 hidden units performs well, and that the best results are obtained by stopping training roughly between epochs 50 and 100. However, there is no single magic number — epoch 73 may be slightly better than epoch 68 for one fold, and worse for another.

Instead of betting on a single stopping epoch, the notebook adopts an **ensemble averaging** strategy: it generates predictions from 50 models (each trained for a slightly different number of epochs) and then averages them.

This has two advantages. First, it reduces variance: a single model may produce noisy predictions, but averaging 50 models stabilizes them. Second, it makes the result robust to the exact choice of early stopping: it does not matter whether the perfect epoch was 67 or 82, because both contribute to the final average.

An Important Point

Note that at each iteration the model is recreated from scratch (`make_model(...)` inside the loop). This means that the 50 models differ not only in the number of training epochs, but also in the random initialization of their weights.

This adds an additional source of diversity to the ensemble, making it even more robust.

What the Final Predictions Represent

The DataFrame `predictions_combined` saved in cell 103 has a precise structure: the index is a MultiIndex (`date`, `ticker`), and the columns are 250 prediction vectors (50 epochs $\times$ 5 folds). Each cell contains the predicted weekly return $\hat{r}_{i,t+1}$ for stock i at week t.

To obtain the model's final prediction for each pair (date, stock), we take the average across the columns:

$$\hat{r}_{i,t+1}^{\text{ensemble}} = \frac{1}{250} \sum_{m=1}^{250} \hat{r}_{i,t+1}^{(m)}$$

This average has two effects. First, it reduces the idiosyncratic noise of each individual model. Second, it makes the result robust to the choice of stopping epoch — it does not matter whether the optimal epoch was 63 or 78, because both contribute.

What the Predictions Are Not

A fundamental point to understand is that these predictions **are not reliable point estimates** of the future return of each individual stock. It would be unrealistic to expect the model to say "Apple will return +2.3% next week." The stock market contains far too much noise for that.

What the model does well is **rank stocks**: among 2,500 equities, those to which it assigns higher predictions do tend, on average, to perform better than those to which it assigns lower predictions. It is a ranking signal, not a level estimate. This is exactly what the IC measures.

How They Are Used in Practice

The autoencoder's predictions are used as inputs for portfolio construction. The typical implementation involves several steps.

Long-short portfolio: each week, stocks are sorted by predicted return. One buys those in the top quintile (top 20%) and short-sells those in the bottom quintile (bottom 20%). The return of this portfolio is the "factor-mimicking portfolio" — if the IC is positive, this portfolio will have a positive expected return.

Sharpe ratio: the key measure of success is the annualized Sharpe ratio of the long-short portfolio. There is a well-known approximate relationship between IC and Sharpe:

$$\mathrm{SR} \approx \mathrm{IC} \times \sqrt{\mathrm{BR}}$$

where BR is "breadth," that is, the number of independent bets (approximately the number of stocks $\times$ the number of periods). With an IC of 0.03 and about 2,500 stocks rebalanced weekly, interesting Sharpe ratios can be obtained.

Performance Evaluation of Alpha Factors with Alphalens

Alphalens Evaluation Pipeline

Setup

- Epoch-averaged predictions $\rightarrow$ single factor score per (date, ticker)
- Aligned with weekly trade prices (Friday, UTC)
- Holding periods: 5D, 10D, 21D
- 5 quintiles ($Q = 5$)

Alphalens Key Function

`get_clean_factor_and_forward_returns` merges factor scores with realised forward returns and assigns quantiles. $\sim$18.5% observations dropped (missing fwd returns).

Diagnostics Produced

1. **Quantile returns**: bar charts, cumulative lines, violin plots
2. **Alpha & Beta**: factor regressed on market return
3. **Information Coefficient**: distribution, time series, monthly heatmap
4. **Turnover**: quantile composition stability, rank autocorrelation
5. **Summary & full tear sheets**

Quantile Returns: Monotonic Cross-Sectional Ordering

Cumulative Returns by Quantile

- **5D horizon**: clear monotonic spread; Q5 outperforms, Q1 underperforms
- **10D**: ordering persists, spread slightly narrows
- **21D**: structure still visible but weaker $\Rightarrow$ signal is primarily **short-term**

Long–Short Spread (Q5 − Q1)

Horizon	Mean Spread (bps)
5D	31.4
10D	16.8
21D	17.8

Alpha & Beta vs. Market

	5D	10D	21D
Ann. Alpha	3.05%	0.70%	3.44%
Beta	0.062	0.054	0.009

Near-zero betas confirm the factor is **orthogonal to the market**: it captures **idiosyncratic**, firm-specific predictive content.

Quantile Returns: Monotonic Cross-Sectional Ordering

Economic Interpretation

The CAE compresses 15 characteristics into a latent score that ranks stocks by expected return. The ranking is stable, monotonic, and market-independent.

Information Coefficient Analysis

IC Summary Statistics

	5D	10D	21D
IC Mean	0.008	0.004	0.009
IC Std	0.053	0.047	0.041
Risk-Adj. IC	0.146	0.095	0.229
t-stat	2.27	1.47	3.54
p-value	0.024	0.142	0.000

Key Observations

- IC positive at **all horizons**
- Statistically significant at 5D (p=0.024) and 21D (p<0.001)

Temporal Dynamics

- Rolling IC oscillates around zero with recurrent **positive phases**
- Sustained strength 2016–2017; brief weakness during late-2018 volatility
- Monthly IC heatmaps: **predominantly blue** (positive), especially at 21D
- No persistent structural breakdowns detected

Information Coefficient Analysis

Interpretation

The CAE factor provides a **statistically significant, temporally stable** predictive signal. Magnitudes (IC ≈ 0.01) are typical for equity factors and sufficient for economically meaningful strategies when applied with high breadth.

Turnover & Signal Persistence

Quantile Turnover

Quantile	5D	10D	21D
Q1	0.496	0.572	0.644
Q3	0.655	0.697	0.734
Q5	0.490	0.568	0.640

Turnover in the 0.49–0.75 range. Extreme quintiles (Q1, Q5) are the **most stable**, suggesting persistent rankings at the tails.

Factor Rank Autocorrelation

Horizon	Mean Autocorr.
5D	0.511
10D	0.399
21D	0.286

Smooth decay from 0.51 to 0.29: the signal persists for **1–3 weeks** before dissipating. This is consistent with short-to-medium-term equity factors.

Turnover & Signal Persistence

Balance Between Adaptability and Persistence

The model updates frequently enough to incorporate new information while maintaining stable rankings, avoiding excessive transaction costs. Turnover spikes coincide with market volatility, not model instability.

Summary of Findings

What the Results Suggest

The conditional autoencoder:

1. **Orders the cross-section coherently**: monotonic Q1–Q5 return spreads at all horizons
2. **Delivers market-orthogonal alpha**: annualised $\alpha \approx 3\%$, $\beta \approx 0$ vs. the market
3. **Produces statistically significant ICs**: t-stat > 2 at 5D and 21D
4. **Remains temporally stable**: no structural breakdowns across 2015–2019
5. **Balances adaptability and persistence**: smooth turnover, rank autocorrelation decays naturally

Conceptual Contribution

The CAE bridges **economic structure** (conditional factor model, no-arbitrage) with **deep learning flexibility** (nonlinear loadings, data-driven factors). It demonstrates that interpretability and complexity are **complementary**, not opposing.

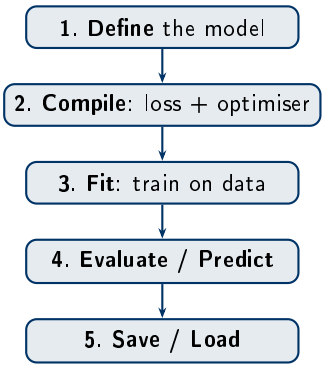
Appendix A
Keras

What Is Keras? — The 5-Step Workflow

Keras is the high-level API of TensorFlow for building and training neural networks. Accessible as `tensorflow.keras`.

Key Characteristics

- **Modular**: models = layers snapped together
- **Two APIs**: Sequential (simple) and Functional (complex)
- **Extensible**: custom layers, losses, callbacks
- Runs on CPU, GPU, and TPU



Sequential vs. Functional API

Sequential — Single Chain of Layers

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,
    Dropout

model = Sequential([
  Dense(64, activation='relu',
        input_shape=(15,)),
  Dropout(0.2),
  Dense(32, activation='relu'),
  Dense(1) # regression output
])
```

One input, one output, no branching.

Functional — Multi-Input / Branching

```
from tensorflow.keras.layers import (
    Input, Dense, BatchNormalization, Dot)
from tensorflow.keras.models import Model

inp_chars = Input((N, P), name='chars')
inp_ret = Input((N,), name='returns')
# Branch 1
x = Dense(8, activation='relu')(inp_chars)
x = BatchNormalization()(x)
betas = Dense(K)(x)
# Branch 2
factors = Dense(K)(inp_ret)
# Merge
out = Dot(axes=(2,1))([betas, factors])

model = Model([inp_chars, inp_ret], out)
```

Core Layers & Activations

Essential Layers

Layer	Purpose
`Dense(n, act)`	Fully connected: $g(Wx+b)$
`Dropout(rate)`	Randomly zeros units (regularisation)
`BatchNorm()`	Normalise activations to $\mu=0, \sigma=1$
`Input(shape)`	Declares entry point (Functional API)
`Dot(axes)`	Dot product of two tensors
`Flatten()`	Collapse dims to 1D vector

Activation Functions

Name	Formula	Use
`relu`	$\max(0, x)$	Hidden layers
`sigmoid`	$\frac{1}{1+e^{-x}}$	Binary output
`softmax`	$\frac{e^{x_i}}{\sum e^{x_j}}$	Multi-class
`linear`	x	Regression

Practical Rule

`relu` in hidden layers.
Output activation depends on the task: `linear` for regression, `sigmoid`/`softmax` for classification.

Compile & Fit

Step 2: Compile

```
model.compile(
  loss='mse', # what to minimise
  optimizer='adam', # how to update weights
  metrics=['mae']) # what to monitor
```

Task	Loss
Regression	`mse`, `mae`
Binary classif.	`binary_crossentropy`
Multi-class	`categorical_crossentropy`

Adam is the default go-to optimiser: adaptive learning rate + momentum.

Step 3: Fit (Train)

```
history = model.fit(
  X_train, y_train,
  epochs=100, # full passes over data
  batch_size=32, # samples per update
  validation_data=(X_val, y_val),
  shuffle=True,
  verbose=1)
```

`history.history` stores train/val loss per epoch.

Diagnosing overfitting via learning curves:
Train loss $\downarrow$, val loss $\uparrow$ $\Rightarrow$ overfitting
Both $\downarrow$ $\Rightarrow$ keep training
Both flat $\Rightarrow$ converged

Evaluate, Predict, Save

```
# Step 4: Evaluate on held-out test data (returns loss + metrics)
test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)

# Step 4: Generate predictions (forward pass only, no gradient)
y_pred = model.predict(X_new)

# Step 5: Save entire model (architecture + weights + optimiser)
model.save('my_model.keras')

# Step 5: Reload later
from tensorflow.keras.models import load_model
model = load_model('my_model.keras')
```

`model.summary()`

Call immediately after building to verify layer shapes and parameter counts. Catches dimensionality errors early.

Weights-Only Save

`model.save_weights('w.h5')` saves only the trained parameters (lighter). Requires re-building the architecture before loading.

Controlling Overfitting

Dropout

```
Dense(64, activation='relu'),
Dropout(0.3), # drop 30% of units
Dense(32, activation='relu'),
```

Randomly disables neurons during training. Off at prediction time.

L_2 Weight Regularisation

```
from tensorflow.keras.regularizers import l2
Dense(64, activation='relu',
      kernel_regularizer=l2(1e-4))
```

Adds $\lambda \|W\|_2^2$ to the loss, shrinking weights.

Batch Normalisation

```
Dense(64, activation='relu'),
BatchNormalization(),
```

Normalises activations per mini-batch. Stabilises training and acts as mild regulariser.

Early Stopping

```
from tensorflow.keras.callbacks \
    import EarlyStopping
es = EarlyStopping(monitor='val_loss',
       patience=10,
       restore_best_weights=True)
model.fit(..., callbacks=[es])
```

Stops training when val loss stops improving.

Callbacks & Multi-Input Models

Useful Callbacks

Callback	What It Does
`EarlyStopping`	Stop when metric plateaus
`ModelCheckpoint`	Save best weights
`ReduceLROnPlateau`	Lower LR on plateau
`TensorBoard`	Interactive log viewer
`CSVLogger`	Metrics to CSV file

```
callbacks = [
  EarlyStopping(monitor='val_loss',
    patience=10,
    restore_best_weights=True),
  ModelCheckpoint('best.keras',
    save_best_only=True)]
model.fit(..., callbacks=callbacks)
```

Multi-Input Pattern

```
inp_a = Input((10,), name='numeric')
inp_b = Input((50,), name='text')
a = Dense(32, activation='relu')(inp_a)
b = Dense(64, activation='relu')(inp_b)
merged = Concatenate()([a, b])
out = Dense(1)(merged)

model = Model([inp_a, inp_b], out)
```

Use the Functional API whenever you need multiple inputs, multiple outputs, or shared layers.

The **conditional autoencoder** uses exactly this pattern: two inputs (`characteristics`, `returns`) merged via `Dot`.

Summary — What to Remember

The 5-Step Mental Model

1. **Define**: stack layers (Sequential or Functional)
2. **Compile**: pick loss + optimiser
3. **Fit**: train, monitor validation
4. **Predict**: forward pass only
5. **Save**: persist for later use

Most-Used Layers

`Input` · `Dense` · `Dropout` · `BatchNormalization` · `Dot` · `Concatenate` · `Flatten`

Practical Tips

- `relu` hidden layers, `linear` regression output
- Start with `adam`, default LR (10^{-3})
- Always use a **validation set**
- Add `EarlyStopping` to save compute
- Call `model.summary()` to check shapes
- Plot learning curves — they tell you everything

Golden Rule

Start simple, inspect `summary()`, plot learning curves, then iterate. Add complexity only when validation loss demands it.

Appendix B
The Alphalens Library

Appendix — Alphalens: Overview

What Is Alphalens?

Alphalens is an open-source Python library originally developed by **Quantopian** for the **performance analysis of predictive alpha factors**. It is designed to answer a single question: *does a given signal actually predict future returns?*

Core Capabilities

- Correlation of signals with subsequent returns
- Profitability of equal- or factor-weighted quantile portfolios
- Factor turnover and implied trading costs
- Factor performance during specific events
- Sector-level breakdowns of all metrics

Ecosystem Integration

- **Zipline**: event-driven backtesting engine that produces the signals Alphalens analyses
- **Pyfolio**: portfolio-level risk and return analytics (Sharpe, drawdowns, etc.)
- Together: **signal** $\rightarrow$ **backtest** $\rightarrow$ **portfolio analysis**

Appendix — Alphalens: Module Structure

Alphalens is organised into four main modules:

Module	Purpose
`alphalens.utils`	Data preparation. The key function `get_clean_factor_and_forward_returns` merges factor signals with forward returns and assigns quantiles. Also handles date alignment, NaN cleaning, and sector mapping.
`alphalens.performance`	Computational engine. Functions to compute mean returns by quantile, the Information Coefficient (IC), factor-weighted returns, cumulative returns, and turnover metrics.
`alphalens.plotting`	Visualisation layer. One plotting function per analysis type: bar charts, line plots, violin plots, heatmaps, and time-series IC charts.
`alphalens.tears`	Tear sheets. High-level convenience functions that combine multiple `performance` computations and `plotting` calls into comprehensive one-call reports.

Appendix — Alphalens: Key Functions Reference

Data Preparation (utils)

```
alphalens.utils.get_clean_factor_and_forward_returns(
    factor, # Series with MultiIndex (date, asset)
    prices, # DataFrame: dates x assets
    periods=(1,5,10), # holding periods in trading days
    quantiles=5, # number of signal bins
    bins=None, # alternative: custom bin edges
    filter_zscore=20, # drop extreme forward returns
    groupby=None) # optional sector grouping
```

Performance Computations (performance)

```
mean_return_by_quantile(factor_data, by_date=False, by_group=False)
factor_information_coefficient(factor_data, group_adjust=False, by_group=False)
factor_returns(factor_data, long_short=True, group_neutral=False)
average_cumulative_return_by_quantile(factor_data, prices, periods_before, ...)
quantile_turnover(quantile_factor, quantile, period=1)
factor_rank_autocorrelation(factor_data, period=1)
```

Appendix — Alphalens: Plotting & Tear Sheets

Plotting Functions (plotting)

```
plot_quantile_returns_bar(mean_return_by_q)
plot_cumulative_returns_by_quantile(mean_return_by_q_daily, period)
plot_quantile_returns_violin(return_by_q)
plot_ic_ts(ic, ax=None) # IC time series
plot_ic_hist(ic, ax=None) # IC histogram
plot_ic_qq(ic, ax=None) # IC Q-Q plot
plot_ic_heatmap(ic, ax=None) # monthly IC heatmap
plot_turnover_table(autocorrelation_data, quantile_turnover)
plot_top_bottom_quantile_turnover(quantile_turnover, period)
plot_factor_rank_auto_correlation(factor_autocorrelation)
```

Tear Sheet Functions (tears)

```
create_summary_tear_sheet(factor_data) # all-in-one report
create_returns_tear_sheet(factor_data) # returns analysis only
create_information_tear_sheet(factor_data) # IC analysis only
create_turnover_tear_sheet(factor_data) # turnover analysis only
create_event_returns_tear_sheet(factor_data, prices, ...) # event study
create_full_tear_sheet(factor_data) # everything combined
```